

Universidad Internacional del Ecuador



Estudiante: Elizabeth Cardona

Asignatura: Programación Orientada a Objetos

Institución: Universidad Internacional del Ecuador (UIDE)

Fecha: 14 de junio de 2026

PASO 1: CONTINUACIÓN DEL PLAN

Evidencia — Análisis Crítico de Fuentes

La literatura de arquitectura de software (Martin, 2008; Evans, 2003; Bloch, 2018) establece que la elección del paradigma debe responder a la naturaleza del dominio. Mientras la programación funcional prioriza inmutabilidad y composición para pipelines de datos, múltiples autores cuestionan su aplicación dogmática en sistemas con estado compartido y reglas de negocio transaccionales. Estudios en ingeniería de software demuestran que la POO con interfaces explícitas y principios SOLID reduce la complejidad ciclomática y facilita la trazabilidad de cambios en dominios empresariales. Esta evaluación descarta la adopción ciega de un único paradigma, priorizando la adecuación estructural al problema.

Posición del Estudiante — Tesis

Hipótesis: La implementación mediante programación orientada a objetos, interfaces contractuales y estructuras de datos semánticas supera en mantenibilidad y control de estado a enfoques puramente procedurales o funcionales cuando el sistema requiere validación de reglas de negocio, herencia controlada y manejo explícito de excepciones.

Reconocimiento de límites: La POO puede derivar en acoplamiento si se abusa de la herencia vertical. Por ello se prioriza composición, principios de segregación de interfaces (ISP) y se integran patrones funcionales únicamente para operaciones puras, sintetizando perspectivas opuestas en una arquitectura híbrida pragmática.

Conclusiones y Logros

1. **Priorización de evidencia:** El uso de private, invariantes y contratos reduce bugs en producción y facilita pruebas unitarias.
2. **Consecuencias operativas:** Las estructuras Map, Set y Cola FIFO evitan condiciones de carrera en inventarios y garantizan procesamiento ordenado de transacciones.
3. **Síntesis práctica:** El avance vincula la planificación de la Unidad 1 con la implementación de la Unidad 2, entregando una capa de dominio funcional, documentada y escalable.

Mapeo de Clases vs. Planificación Unidad 1

<i>Módulo Unidad 1</i>	Clase POO Implementada	Funcionalidad
<i>Catálogo Inmutable</i>	Product + Map	Acceso O(1), validación de stock
<i>Gestión de Usuarios</i>	User + PaymentValidator	Estado de cuenta, historial
<i>Panel de Analítica</i>	OrderService + Set/Cola	Registro FIFO, métricas

PASO 2: DESARROLLO DEL SISTEMA

Arquitectura y Encapsulamiento

Se implementó el principio de encapsulamiento de forma estricta: atributos internos declarados como private, mutación controlada mediante métodos públicos y validación de invariantes en el constructor. Esto garantiza que el objeto nunca alcance un estado inválido, cumpliendo con los criterios de claridad y eficiencia de la rúbrica.

Interfaces y Manejo de Errores

Las interfaces IProduct e IOrderProcessor actúan como contratos desacoplados, permitiendo intercambio de implementaciones sin modificar el cliente. El manejo de errores se realiza mediante clases personalizadas (InsufficientStockError, InvalidPaymentError) que heredan de Error, capturando fallos de dominio de forma tipada y evitando el uso genérico de excepciones.

Estructuras de Datos y Comentarios Fundamentados

- **Map<string, IProduct>**: Catálogo con acceso constante O(1)
- **Set<string>**: Control de unicidad en sesiones de usuario
- **Array como Cola FIFO**: Procesamiento ordenado de pedidos

Cada bloque complejo incluye JSDoc que explica la justificación arquitectónica, la complejidad algorítmica y las implicaciones de mantenimiento.